C. Wrong Binary Search

time limit per test 2 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1300 | Tags: binary search, constructive algorithms | https://codeforces.com/problemset/problem/2146/C

You are given an integer n and a binary string[□] s of length n.

For a permutation[†] p of length n and an integer x, we define find(x) as in the following pseudocode:

```
function find(int x):
    l := 1
    r := n
    while l <= r:
        let m be a random integer between l and r (inclusive)
        if p[m] == x
            return m
        if p[m] > x:
            r := m - 1
        else:
            l := m + 1
    return undefined     // not found
```

We call an integer x ($1 \le x \le n$) *stable* if and only if find(x) is **always** not undefined and $p_{\text{find}(x)} = x$ **always** holds, no matter how the value of m is selected in the process of the pseudocode.

You have to construct a permutation p of length n, such that:

- For each $1 \le i \le n$, the integer i is *stable* **if and only if** s_i=mathtt1.

Or determine that no such permutation exists.

[□]A binary string is a string where each character is either mathtt0 or mathtt1.

[†]A permutation of length n is an array consisting of n distinct integers from 1 to n in arbitrary order. For example, [2,3,1,5,4] is a permutation, but [1,2,2] is not a permutation (2 appears twice in the array), and [1,3,4] is also not a permutation (n=3 but there is 4 in the array).

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \le n \le 2 \cdot 10^5$) — the length of the permutation.

The second line contains the binary string s of length n (s_i=mathtt0 or mathtt1).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case:

- If no such permutation exists, print "NO" in the only line of output.
- Otherwise, print "YES" in the first line of output. Then output n distinct integers $p_1, p_2, \ldots, p_n$ ($1 \le p_i \le n$) in the second line — the permutation you constructed.

You can output the token in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

If there are multiple answers, you may print any of them.

Example

Input
6 3 111 5 00000 5 10100 7 0010000 11 00001001100 12 011100010000

Output
YES 1 2 3 YES 2 4 3 5 1 NO YES 2 1 3 5 7 6 4 YES 2 1 4 3 5 7 6 8 9 11 10 NO

Note

In the first test case, we have constructed p=[1,2,3]. Take find(2) as an example:

- In the beginning, l=1 and r=3;
- We will select m as a random integer between l=1 and r=3.
- If we choose m=1:
- p_1=1 lt 2, so l is set to m + 1 = 2. Now l=2 and r=3;
- We will select m as a random integer between l=2 and r=3.
- If we choose m=2: p_2=2, so the return value is 2.
- If we choose m=3: p_3=3 gt 2, so r is set to m-1=2. Now l=2 and r=2, so we can only select m=2. p_2=2, so the return value is 2.
- If we choose m=2:
- p_2=2, so the return value is 2.
- If we choose m=3, the process is similar to that when m=1 is chosen. The return value is always 2.
- Thus, no matter how m is selected during the process, the return value of find(2) is always 2.

So $p_{\text{find(2)}} = p_2 = 2$ always holds, and the integer 2 is *stable*.

Similarly, we can show that integers 1 and 3 are also *stable*. Thus, the permutation p = [1, 2, 3] is a valid answer.

In the second test case, we have constructed p=[2,4,3,5,1]. Take find(3) as an example:

- In the beginning, l=1 and r=5;
- We will select m as a random integer between l=1 and r=5. Let's choose m=2;
- p_2=4 gt 3, so r is set to m-1=1. Now l=1 and r=1;
- We will select m as a random integer between l=1 and r=1. We can only choose m=1;
- p_1=2 lt 3, so l is set to m + 1 = 2. Now l=2 and r=1;
- Since l gt r, the loop is exited and the return value is undefined.

Thus, the return value of find(3) is undefined, so the integer 3 is not *stable*.

Similarly, we can show that integers 1, 2, 4, and 5 are also not *stable*. Thus, p = [2, 4, 3, 5, 1] is a valid answer.

Some other permutations, such as p=[5,4,3,2,1] and p=[3,5,1,4,2], are also valid answers. You may print any of them.

In the third test case, it can be proven that no such permutation exists.